markdown

🔐 CATEGORY 8: Security & Permissions — Complete Deep Dive

8.1 `sudo` Internals — `/etc/sudoers`, `NOPASSWD`, `visudo`

🔷 What sudo is in simple terms

`sudo` (**s**uper**u**ser **do**) lets a permitted user run a command **as another user** (usually root) without knowing that user's password. It's the controlled delegation of privilege — instead of giving everyone the root password, you grant specific commands to specific users.

🔷 How sudo works internally

```
User runs: sudo systemctl restart nginx
                 │
                 ▼
          /usr/bin/sudo (setuid root binary)
                 │
          1. Checks /etc/sudoers (and /etc/sudoers.d/*)
          2. Does this user have permission for this command?
          3. Prompts for USER'S OWN password (not root's)
          4. Verifies password against PAM
          5. Checks timestamp cache (~/.sudo_as_admin_successful)
                 │
          If permitted:
                 │
          ▼
          fork() → setuid(0) → exec("systemctl restart nginx")
                 │
          Logs the action to /var/log/auth.log or journald

sudo's security model:
  ✓ Root password never shared
  ✓ Every sudo action is logged with username + command
  ✓ Granular: grant specific commands only
  ✓ Time window: re-ask password after timeout (default 15 min)
```

🔷 `/etc/sudoers` — The permission file

```
# NEVER edit /etc/sudoers directly — use visudo instead!
# visudo: locks the file, validates syntax before saving
# Direct edit mistake → can lock everyone out of sudo!

sudo visudo          # Opens in $EDITOR with lock + syntax check
sudo visudo -f /etc/sudoers.d/myapp   # Edit a drop-in file

# ── File format ──────────────────────────────────────────────

# Syntax: WHO  WHERE=(AS_WHOM)  WHAT

# user/group  host=(runas_user:runas_group)  command

# ── Basic examples ───────────────────────────────────────────

# Allow alice to run ANY command as root on ANY host (full sudo)
alice   ALL=(ALL:ALL)   ALL

# Allow alice to run specific commands only
alice   ALL=(root)      /usr/bin/systemctl, /usr/bin/journalctl

# Allow a group (note the % prefix)
%wheel  ALL=(ALL:ALL)   ALL
%admin  ALL=(ALL)       ALL

# NOPASSWD — no password prompt (for automation/scripts)
deploy  ALL=(root)      NOPASSWD: /usr/bin/systemctl restart nginx

# Combined: specific commands, no password
jenkins ALL=(root)      NOPASSWD: /usr/bin/docker, /usr/bin/kubectl

# Run as a specific non-root user
alice   ALL=(www-data)  /usr/bin/php

# ── Field meanings ───────────────────────────────────────────

# WHO:
#   username        → specific user
#   %groupname      → all members of this group
#   #UID            → user with this UID
#   %#GID           → members of group with this GID

# WHERE (host):
#   ALL             → any host (standard when sudoers is local)
#   server01        → only on host named server01
#   (rarely used — sudoers is per-host)

# AS_WHOM (runas):
#   ALL             → any user
#   root            → only as root
#   (alice)         → only as alice
#   (ALL:ALL)       → any user:any group
```

```
# WHAT (commands):
#   ALL             → any command
#   /path/to/cmd    → specific binary (full path required!)
#   /path/to/cmd *  → binary with any arguments
#   /path/to/cmd ""  → binary with NO arguments

# ── Command argument control ──────────────────────────────────────

# Allow systemctl but only restart/status, not stop/disable
alice  ALL=(root)  /usr/bin/systemctl restart *, /usr/bin/systemctl status
*

# Dangerous: allow with any args (can be abused!)
alice  ALL=(root)  /usr/bin/vim    # vim can run shell commands!
# Someone with sudo vim can: :!/bin/bash → get root shell

# ── Aliases for cleaner rules ─────────────────────────────────────

# User aliases
User_Alias  WEBTEAM = alice, bob, charlie
User_Alias  DBTEAM  = dave, eve

# Command aliases
Cmnd_Alias  WEB_CMDS = /usr/bin/systemctl restart nginx, \
                       /usr/bin/systemctl reload nginx,  \
                       /usr/bin/certbot
Cmnd_Alias  DB_CMDS  = /usr/bin/systemctl restart postgresql, \
                       /usr/bin/pg_dump

# Host aliases
Host_Alias  WEBSERVERS = web01, web02, web03
Host_Alias  DBSERVERS  = db01, db02

# Now use them:
WEBTEAM  WEBSERVERS=(root)  WEB_CMDS
DBTEAM   DBSERVERS=(root)   DB_CMDS

# ── /etc/sudoers.d/ — Drop-in directory ──────────────────────────

# Modern approach: never edit /etc/sudoers directly
# Add files to /etc/sudoers.d/ — each is automatically included

sudo visudo -f /etc/sudoers.d/deploy
# Contents:
# deploy  ALL=(root)  NOPASSWD: /usr/bin/systemctl restart *, \
#                               /usr/bin/docker

# ⚠ File permissions must be 0440
sudo chmod 0440 /etc/sudoers.d/deploy

ls -la /etc/sudoers.d/
# -r--r----- 1 root root  89 Mar 10 14:00 deploy
# -r--r----- 1 root root 156 Mar 10 12:00 jenkins
```

🔷 sudo command usage

```
# Run command as root
sudo systemctl restart nginx

# Run command as specific user
sudo -u www-data php artisan migrate

# Run command as specific user AND group
sudo -u alice -g developers ls /home/alice

# Open a root shell
sudo -i           # Login shell (loads root's profile, ~root)
sudo -s           # Non-login shell (keeps current environment mostly)
sudo bash         # Direct bash as root

# Edit a file safely as root (uses $SUDO_EDITOR)
sudo -e /etc/nginx/nginx.conf     # sudoedit — safer than sudo vim
sudoedit /etc/nginx/nginx.conf    # Same

# List what current user can sudo
sudo -l
# User alice may run the following commands on server01:
#     (root) /usr/bin/systemctl restart nginx
#     (root) NOPASSWD: /usr/bin/journalctl

# Check if a specific command is allowed
sudo -l -U alice /usr/bin/systemctl

# Invalidate sudo timestamp (force re-authentication)
sudo -k

# Run with specific environment preserved
sudo -E mycommand    # Preserve current environment variables

# ── Sudo logging ──────────────────────────────────────────────

# Every sudo command is logged
tail -f /var/log/auth.log | grep sudo
# Mar 10 14:32:01 server01 sudo: alice : TTY=pts/0 ; PWD=/home/alice ;
#                          USER=root ; COMMAND=/usr/bin/systemctl restart
nginx

# With journald
journalctl _COMM=sudo -f
```

🔷 Short crisp interview answer

> "`sudo` is a setuid-root binary that reads `/etc/sudoers` to determine if the requesting user is allowed to run a specific command as another user. Always edit with `visudo` — it validates syntax and prevents lockout. The key fields: `alice ALL=(root) NOPASSWD: /usr/bin/systemctl` means alice on any host can run systemctl as root without a password prompt. I use `/etc/sudoers.d/` drop-in files for application-specific grants, and `sudo -l` to audit what a user can do. Every sudo invocation is logged to `/var/log/auth.log` with the username, command, and working directory."

⚠ Gotchas

```
# GOTCHA 1: Never edit /etc/sudoers directly
# Syntax error → everyone loses sudo access
# Fix: boot to single-user mode, mount filesystem, fix manually
# Prevention: ALWAYS use visudo

# GOTCHA 2: Full path required in sudoers
alice  ALL=(root)  systemctl    # ❌ won't match — no full path
alice  ALL=(root)  /usr/bin/systemctl  # ✅ correct

# GOTCHA 3: sudo vim / sudo less / sudo man = root shell
# These editors/pagers can execute shell commands!
# sudo vim → :!/bin/bash → root shell
# NEVER grant sudo to editors unless intended
# Use sudoedit / sudo -e instead

# GOTCHA 4: sudo -i vs sudo -s
# sudo -i: full login shell, loads /root/.bashrc, /root/.profile
#          PATH and HOME change to root's
# sudo -s: shell with root UID but current user's environment
#          Can cause "command not found" for root-only paths

# GOTCHA 5: NOPASSWD timing
# Without NOPASSWD: sudo caches auth for 15 min (default)
# With NOPASSWD: never prompts — good for automation, risky if account
compromised
# Set: Defaults timestamp_timeout=30  (30 min cache)
# Set: Defaults timestamp_timeout=0   (always ask)
# Set: Defaults timestamp_timeout=-1  (never expire once authenticated)

# GOTCHA 6: sudo with wildcards is dangerous
alice  ALL=(root)  /usr/bin/systemctl * restart
# This allows: sudo systemctl evil_service restart
# Better: list specific services explicitly
```

8.2 PAM — Pluggable Authentication Modules

🔷 What PAM is in simple terms

PAM (**Pluggable Authentication Modules**) is the **authentication middleware** of Linux. Instead of every application implementing its own authentication, they all delegate to PAM. PAM then runs a configurable stack of modules — checking passwords, enforcing policies, logging, creating home directories, and more.

🔷 How PAM works

```
Application (ssh, sudo, login, su...)
           │
           │ calls PAM via libpam.so
           ▼
     ┌──────────────────────────────────┐
     │          PAM Framework           │
     │                                  │
     │  Reads: /etc/pam.d/<service>     │
     │                                  │
     │  Executes module stack:          │
     │   ┌──────────────────────────┐   │
     │   │  auth     pam_unix.so    │ ─► check /etc/shadow password
     │   │  auth     pam_tally2.so  │ ─► check failed login count
     │   │  account  pam_nologin.so │ ─► check /etc/nologin
     │   │  account  pam_time.so    │ ─► check time restrictions
     │   │  session  pam_limits.so  │ ─► apply /etc/security/limits.conf
     │   │  session  pam_motd.so    │ ─► show message of the day
     │   └──────────────────────────┘   │
     └──────────────────────────────────┘
           │
           │ Returns: success or failure
           ▼
    Application proceeds or rejects

Four management groups:
  auth      → verify identity (password, token, fingerprint)
  account   → check account validity (expiry, time, lockout)
  session   → setup/teardown session (home dir, limits, logging)
  password  → change credentials
```

🔷 PAM configuration files

```
# ── Service files ──────────────────────────────────────────────
ls /etc/pam.d/
# common-auth      ← shared by many services
# common-account
# common-password
# common-session
# sshd             ← SSH specific
```

```
# sudo             ← sudo specific
# login            ← console login
# su               ← su command

# ── Line format ──────────────────────────────────────────────────
# type  control  module-path  module-arguments

# type:    auth | account | session | password
# control: required | requisite | sufficient | optional | include
# module:  /lib/security/pam_unix.so (or short name: pam_unix.so)
# args:    module-specific options

# ── Control flags explained ──────────────────────────────────────

# required:
#   Module MUST succeed for overall success.
#   But: if it fails, PAM still runs remaining modules (for security —
attacker
#   doesn't know which module failed)
#   Failure of a required module: entire stack fails (but keeps running)

# requisite:
#   Module MUST succeed.
#   If it fails: STOP immediately, return failure.
#   Difference from required: stops on first failure (faster, but reveals
failure point)

# sufficient:
#   If this module SUCCEEDS (and no prior required module failed):
#   STOP processing, return success immediately.
#   Common use: allow root login without password check

# optional:
#   Module result is IGNORED for pass/fail decision.
#   Used for side effects (logging, home dir creation)

# include:
#   Pull in another PAM config file.
#   @include means the same thing.

# ── Example: /etc/pam.d/sshd ─────────────────────────────────────

cat /etc/pam.d/sshd
# @include common-auth              ← use common auth stack
# account  required     pam_nologin.so   ← block login if /etc/nologin
exists
# account  include      common-account
# session  [success=ok ignore=ignore module_unknown=ignore default=bad]
pam_selinux.so close
# session  required     pam_loginuid.so  ← set audit login UID
# session  include      common-session
# session  optional     pam_motd.so      ← show /etc/motd
# session  optional     pam_mail.so      ← notify about mail
```

```
# ── Example: /etc/pam.d/common-auth ──────────────────────────────

cat /etc/pam.d/common-auth
# auth  [success=1 default=ignore]  pam_unix.so  nullok_secure
# auth  requisite                   pam_deny.so
# auth  required                    pam_permit.so
# auth  optional                    pam_cap.so
```

🔷 Important PAM modules

```
# ── pam_unix.so — standard UNIX authentication ──────────────────
# Checks /etc/passwd and /etc/shadow
# auth     required  pam_unix.so  shadow nullok try_first_pass
#                                  ──────  ─────  ──────────────
#                                  use shadow   empty   try stored
password first
#                                  file         pwd OK

# ── pam_tally2.so / pam_faillock.so — account lockout ───────────
# Lock account after N failed attempts
auth     required  pam_faillock.so preauth silent deny=5 unlock_time=900
auth     required  pam_unix.so
auth     [default=die] pam_faillock.so authfail deny=5 unlock_time=900
account  required  pam_faillock.so
# deny=5: lock after 5 failures
# unlock_time=900: unlock after 15 minutes

# Check locked accounts
faillock --user alice
# alice:
# When                Type  Source
Valid
# 2026-03-10 14:23:01 RHOST 192.168.1.100
V
# 2026-03-10 14:23:05 RHOST 192.168.1.100
V

# Unlock account manually
faillock --user alice --reset

# ── pam_pwquality.so — password complexity ──────────────────────
# /etc/security/pwquality.conf
# minlen = 12        # Minimum 12 characters
# minclass = 3       # Must have 3 character classes
(upper/lower/digit/special)
# maxrepeat = 3      # No more than 3 repeated chars
# dcredit = -1       # Must have at least 1 digit
# ucredit = -1       # Must have at least 1 uppercase
# lcredit = -1       # Must have at least 1 lowercase
# ocredit = -1       # Must have at least 1 special char
```

```
# dictcheck = 1      # Check against dictionary

password required pam_pwquality.so retry=3

# ── pam_limits.so — resource limits ──────────────────────────────
# Applied via /etc/security/limits.conf
session required pam_limits.so

# /etc/security/limits.conf format:
# domain     type  item   value
# alice      hard  nproc  100       # Hard limit: max 100 processes
# @devteam   soft  nofile 65536     # Soft limit: max 65536 open files
# *          hard  core   0         # Disable core dumps for all
# nginx      -     nofile 65536     # Both soft and hard for nginx user

# ── pam_time.so — time-based access control ──────────────────────
# /etc/security/time.conf
# services;ttys;users;times
# login;tty*;!alice;Al0800-1800   # alice cannot login on weekdays 8am-6pm
# sshd;*;contractors;Wk0900-1700  # contractors only Mon-Fri 9am-5pm

account required pam_time.so

# ── pam_mkhomedir.so — auto-create home directory ────────────────
# Used with LDAP/AD users who don't have local home dirs
session optional pam_mkhomedir.so skel=/etc/skel umask=0077

# ── pam_duo.so / pam_google_authenticator.so — MFA ───────────────
# Add two-factor authentication
auth required pam_google_authenticator.so
# User must enter TOTP code after password
```

🔷 Debugging PAM

```
# PAM logs to auth.log / secure
tail -f /var/log/auth.log | grep -i pam

# Common log entries:
# pam_unix(sshd:auth): authentication failure; logname= uid=0 euid=0
#   tty=ssh ruser= rhost=1.2.3.4  user=alice
# pam_faillock(sshd:auth): Consecutive login failures for user alice
#   account temporarily locked out

# Enable PAM debug logging (for specific service)
# Add to /etc/pam.d/sshd:
# auth required pam_debug.so   ← ONLY for debugging, remove after!

# Test PAM authentication manually
pamtester sshd alice authenticate
# pamtester: successfully authenticated alice
```

🔷 Short crisp interview answer

> "PAM is Linux's authentication middleware — every service (sshd, sudo, login, su) delegates authentication to PAM instead of implementing it themselves. PAM reads `/etc/pam.d/<service>` and runs a stack of modules in order. The control flags matter: `required` must pass but keeps running others; `requisite` stops immediately on failure; `sufficient` stops on success. Key modules I configure in production: `pam_faillock` for account lockout after brute-force attempts, `pam_pwquality` for password complexity, and `pam_limits` to apply resource limits from `/etc/security/limits.conf`."

⚠ Gotchas

```
# GOTCHA 1: Misconfigured PAM = nobody can log in
# A broken pam.d/sshd can lock out everyone via SSH
# Prevention: always test with a second session before closing current

# GOTCHA 2: required vs requisite timing attack
# required: always runs all modules — attacker can't tell WHICH module
failed
# requisite: stops early — timing difference can reveal failure point
# For auth: prefer required over requisite for security

# GOTCHA 3: pam_limits only applies to PAM sessions
# Limits in /etc/security/limits.conf only take effect for PAM logins
# Systemd services have their own limits (LimitNOFILE= in service file)
# Docker containers also ignore limits.conf

# GOTCHA 4: include vs @include
# include: stops processing current file if included file returns
ignore/die
# @include: processes included file in current stack, then continues
# Modern systems use @include — safer behavior

# GOTCHA 5: Order matters in PAM stack
auth  required   pam_faillock.so preauth   # Must be FIRST
auth  required   pam_unix.so               # Then password check
auth  [default=die] pam_faillock.so authfail  # Must be AFTER pam_unix
# Wrong order: lockout never triggers or always triggers
```

8.3 ACLs — `getfacl`, `setfacl` — Beyond Standard Permissions

🔷 What ACLs are

Standard Unix permissions allow exactly three permission sets: owner, group, and others. **ACLs (Access Control Lists)** extend this by allowing **per-user and per-group permissions** on any file — without changing ownership or adding users to groups.

🔷 Standard permissions vs ACLs

```
Standard permissions (9 bits):
  -rwxr-x--- alice devteam  file.txt
   ───────────────
   owner: rwx  (alice: full access)
   group: r-x  (devteam: read + execute)
   other: ---  (everyone else: no access)

   Problem: How do you give bob read access WITHOUT adding
   him to devteam group?
   → You can't with standard permissions alone.

With ACL:
  -rwxr-x---+ alice devteam  file.txt
              ↑
              + means ACL is set (extra entries exist)

  alice:  rwx   (owner — same as before)
  devteam:r-x   (group — same as before)
  bob:    r--   ← NEW: bob specifically gets read access
  other:  ---   (unchanged)

  Now bob can read the file without joining devteam.
```

🔷 `getfacl` — View ACLs

```
# View ACL on a file
getfacl /var/www/html/config.php
# file: var/www/html/config.php
# owner: www-data
# group: www-data
# user::rw-          ← owner permissions
# user:deploy:r--    ← ACL entry for user 'deploy'
# group::r--         ← owning group permissions
# group:devops:rw-   ← ACL entry for group 'devops'
# mask::rw-          ← effective permission mask
# other::---         ← others permissions

# Recursive
getfacl -R /var/www/

# Check ACL on directory
getfacl /shared/project/
```

```
# file: shared/project/
# owner: alice
# group: engineering
# user::rwx
# user:bob:r-x         ← bob can read/traverse
# user:charlie:rwx     ← charlie has full access
# group::r-x
# group:contractors:r-- ← contractors: read only, no execute
# mask::rwx
# other::---
# default:user::rwx         ← default ACL for new files
# default:user:bob:r-x      ← bob's access on new files too
# default:group::r-x
# default:mask::rwx
# default:other::---
```

🔷 `setfacl` — Set ACLs

```
# ── Grant access ──────────────────────────────────────────────

# Give user bob read access to a file
setfacl -m u:bob:r-- /var/www/html/config.php
# -m = modify
# u:bob:r-- = user bob, read-only

# Give group contractors read+execute
setfacl -m g:contractors:r-x /shared/scripts/

# Multiple entries at once
setfacl -m u:bob:r--,u:charlie:rw-,g:devops:rw- /var/log/app.log

# ── Remove access ─────────────────────────────────────────────

# Remove bob's ACL entry
setfacl -x u:bob /var/www/html/config.php
# -x = remove specific entry

# Remove ALL ACL entries (revert to standard permissions)
setfacl -b /var/www/html/config.php
# -b = remove all ACL entries (including mask)

# Remove default ACLs only
setfacl -k /shared/project/

# ── Recursive ACL setting ─────────────────────────────────────

# Apply to all files/dirs recursively
setfacl -R -m u:bob:r-x /shared/project/
# ⚠ This sets same ACL on files and dirs — usually wrong!
# Files shouldn't need execute; dirs need execute to traverse
```

```
# Better: set different ACLs for files vs directories
# For files:
find /shared/project/ -type f -exec setfacl -m u:bob:r-- {} +

# For directories:
find /shared/project/ -type d -exec setfacl -m u:bob:r-x {} +

# ── Default ACLs — inherited by new files ──────────────────────────

# Problem: you grant bob access to /shared/project/ recursively,
# but alice creates a new file → bob loses access to it!

# Solution: default ACLs — set on directories, inherited by new files

setfacl -m d:u:bob:r-- /shared/project/
setfacl -m d:g:devops:rw- /shared/project/
# d: prefix = default ACL

# Combined: existing files + default for new files
setfacl -R -m u:bob:r--,d:u:bob:r-- /shared/project/

# View default ACLs
getfacl /shared/project/ | grep default
# default:user::rwx
# default:user:bob:r--
# default:group::r-x
# default:mask::r-x
# default:other::---

# ── The mask — effective permissions ───────────────────────────────

# The mask limits the MAXIMUM effective permission of ACL entries
# (does not affect the owner or other)

getfacl file.txt
# user::rw-
# user:bob:rwx    ← bob requested rwx
# group::r--
# mask::r--       ← but mask is r-- only
# other::---

# Effective permissions:
# bob's effective: rwx AND r-- = r--  (mask limits!)
# Check effective permissions:
getfacl file.txt | grep effective
# user:bob:rwx            #effective:r--

# The mask is automatically updated when you setfacl
# Recalculate mask manually:
setfacl --recalculate-mask file.txt

# ── Copy ACLs between files ────────────────────────────────────────
```

```
# Copy ACL from one file to another
getfacl source_file.txt | setfacl --set-file=- target_file.txt

# Backup and restore ACLs
getfacl -R /shared/ > /backup/acl_backup.txt
setfacl --restore=/backup/acl_backup.txt

# ── Filesystem must support ACLs ──────────────────────────────────────

# Check if ACL is enabled
mount | grep /shared
# /dev/sdb1 on /shared type ext4 (rw,relatime,acl)
#                                            ↑ acl option present

# Enable ACL on existing mount (usually already default on ext4/xfs)
sudo mount -o remount,acl /shared
# Add to /etc/fstab:
# UUID=xxx  /shared  ext4  defaults,acl  0 2

# Check if filesystem supports ACL
tune2fs -l /dev/sdb1 | grep "Default mount options"
# Default mount options: user_xattr acl   ← acl present = supported
```

🔷 Short crisp interview answer

> "Standard Unix permissions only allow owner, group, and others — ACLs extend this to per-user and per-group permissions on any file. `setfacl -m u:bob:r-- file` gives bob read access without changing group membership. The `+` in `ls -la` output means an ACL is present. For directories, I always set **default ACLs** with `setfacl -m d:u:bob:r--` so new files inherit the ACL — otherwise new files created by other users won't have the ACL applied. The `mask` field limits the maximum effective permission for all ACL entries (except owner/others)."

⚠ Gotchas

```
# GOTCHA 1: Files created inside ACL dir don't inherit ACL
mkdir /shared
setfacl -m u:bob:r-x /shared   # Sets ACL on /shared itself
touch /shared/newfile.txt       # bob CANNOT read this!
# Fix: set default ACL: setfacl -m d:u:bob:r-- /shared

# GOTCHA 2: The mask surprises
setfacl -m u:bob:rwx file.txt
# bob's effective may not be rwx if mask is restrictive
# Check: getfacl file.txt | grep "effective"

# GOTCHA 3: cp and tar behavior
cp file.txt copy.txt     # ACLs NOT preserved by default
cp -p file.txt copy.txt  # -p preserves permissions but not ACLs
```

```
cp --preserve=all file.txt copy.txt  # Preserves ACLs

tar czf backup.tar.gz /shared     # Loses ACLs
tar --xattrs czf backup.tar.gz /shared  # Preserves ACLs

# GOTCHA 4: chmod affects the mask
chmod 750 file.txt  # Resets the mask to 750!
# This can silently restrict all ACL entries
# After chmod, re-apply setfacl if needed

# GOTCHA 5: Not all filesystems support ACLs
# VFAT, FAT32: no ACL support
# NFS: ACLs may not propagate (depends on NFS version and server)
# tmpfs: no ACL support

# GOTCHA 6: The three-step ACL pattern for shared directories:
# Fix base permissions with find + exec chmod separately for files and
dirs
# Set ACL on existing content with find + exec setfacl
# Always set default ACL with d: prefix for new file inheritance
# Missing step 3 is the #1 real-world ACL mistake.
```

8.4 SELinux / AppArmor — MAC, Contexts, Modes, Troubleshooting

🔷 What MAC is

Standard Linux permissions are **DAC** (Discretionary Access Control) — the file owner decides permissions. **MAC** (Mandatory Access Control) adds a second layer: the **kernel enforces a policy** regardless of what the owner says.

```
DAC (standard permissions):
  Owner controls who can access their files.
  Root can override everything.
  Compromised process running as www-data can access any www-data-owned
file.

MAC (SELinux/AppArmor):
  Kernel policy says: nginx can ONLY access /var/www/html, /var/log/nginx.
  Even if nginx is compromised, it CANNOT access /etc/shadow, /home/*,
/etc/passwd.
  Root cannot override MAC policy (without changing the policy itself).

Container escape example:
  Without MAC: container escape → access to host filesystem
  With MAC:    container escape → still confined to container's
SELinux/AppArmor context
```

🔷 SELinux — Red Hat / RHEL / CentOS / Fedora

```
# ── Modes ──────────────────────────────────────────────────────────

# Enforcing:  MAC policy is enforced. Violations are denied AND logged.
# Permissive: Policy is NOT enforced. Violations are logged ONLY.
#             (great for debugging — see what WOULD be denied)
# Disabled:   SELinux completely off (requires reboot to change)

# Check current mode
getenforce
# Enforcing

sestatus
# SELinux status:                 enabled
# SELinuxfs mount:                /sys/fs/selinux
# SELinuxfs status:               enabled
# Loaded policy name:             targeted
# Current mode:                   enforcing
# Mode from config file:          enforcing
# Policy MLS status:              enabled
# Policy deny_unknown status:     denied
# Max kernel policy version:      33

# ── Temporarily change mode ────────────────────────────────────────

# Switch to permissive (no reboot needed, temporary)
sudo setenforce 0    # Permissive
sudo setenforce 1    # Enforcing

# ── Permanent mode ─────────────────────────────────────────────────

# /etc/selinux/config (or /etc/sysconfig/selinux)
cat /etc/selinux/config
# SELINUX=enforcing        ← change to permissive or disabled
# SELINUXTYPE=targeted     ← policy type

# ⚠ Changing to/from disabled requires reboot AND filesystem relabel
# Relabel: touch /.autorelabel  then  reboot
# (relabeling assigns correct SELinux contexts to all files)

# ── SELinux contexts ───────────────────────────────────────────────

# Every file, process, port, and user has a CONTEXT (security label)
# Format: user:role:type:level
#         system_u:object_r:httpd_sys_content_t:s0

# View file context
ls -Z /var/www/html/
# -rw-r--r--. root root system_u:object_r:httpd_sys_content_t:s0
```

```
index.html
#                       ──────────────────────────────────────────
#                       user:role:type:level

ls -Z /etc/shadow
# ----------. root root system_u:object_r:shadow_t:s0 /etc/shadow
#                                          ────────── ← shadow_t context

# View process context
ps auxZ | grep nginx
# system_u:system_r:httpd_t:s0  www-data  1234  ...  nginx

# View port context
semanage port -l | grep http
# http_port_t    tcp  80, 443, 488, 8008, 8009, 8443
# http_cache_port_t  tcp  3128, 8080, 8118...

# ── Common SELinux booleans ──────────────────────────────────────

# Booleans: switches that adjust policy without rewriting it
getsebool -a | grep httpd
# httpd_can_network_connect --> off
# httpd_can_network_connect_db --> off
# httpd_can_sendmail --> off
# httpd_enable_cgi --> on
# httpd_read_user_content --> off

# Allow nginx/httpd to connect to network (e.g., for proxy to backend)
sudo setsebool -P httpd_can_network_connect on
# -P = persistent (survives reboot)

# Allow httpd to connect to databases
sudo setsebool -P httpd_can_network_connect_db on

# Allow nginx to serve from home directories
sudo setsebool -P httpd_read_user_content on

# ── Fixing "permission denied" with SELinux ──────────────────────

# Scenario: nginx can't serve files from /data/www/
# Step 1: Check it's SELinux (not regular permissions)
sudo setenforce 0   # Set permissive
curl http://localhost   # If it works now → it WAS SELinux

# Step 2: Check the context
ls -Z /data/www/
# -rw-r--r--. root root unconfined_u:object_r:default_t:s0 index.html
#                                                 ────────← WRONG! Should
be httpd_sys_content_t

# Step 3: Fix the context
sudo semanage fcontext -a -t httpd_sys_content_t "/data/www(/.*)?"
sudo restorecon -Rv /data/www/
# restorecon: relabeled /data/www/index.html from default_t to
```

```
  httpd_sys_content_t

  # Step 4: Re-enable enforcing
  sudo setenforce 1
  curl http://localhost   # Now works!

  # ── Alternative: copy context from working directory ──────────────────
  sudo semanage fcontext -a -e /var/www/html /data/www
  sudo restorecon -Rv /data/www/

  # ── audit2allow — generate policy from denials ────────────────────────

  # "What SELinux denials happened recently?"
  sudo ausearch -m AVC -ts recent
  # type=AVC msg=audit(1710012345.678:890): avc: denied { read } for
  # pid=1234 comm="nginx" name="config.php" dev="sda1" ino=12345
  # scontext=system_u:system_r:httpd_t:s0
  # tcontext=system_u:object_r:etc_t:s0 tclass=file permissive=0

  # Generate policy to allow it
  sudo ausearch -m AVC -ts recent | audit2allow
  # #============= httpd_t ==============
  # allow httpd_t etc_t:file read;

  # Generate and install policy module
  sudo ausearch -m AVC -ts recent | audit2allow -M mynginxpolicy
  sudo semodule -i mynginxpolicy.pp

  # ── Check if a denial happened (troubleshoot 403/connection refused) ──
  sudo ausearch -m AVC -ts today | grep nginx
  sudo grep "avc: denied" /var/log/audit/audit.log | tail -20
```

🔷 AppArmor — Ubuntu / Debian / SUSE

```
  # AppArmor uses PATH-based profiles instead of context labels
  # Simpler than SELinux but less fine-grained

  # ── Check status ──────────────────────────────────────────────────────

  sudo aa-status
  # apparmor module is loaded.
  # 27 profiles are loaded.
  # 25 profiles are in enforce mode.
  # 2 profiles are in complain mode.
  #   /usr/bin/man
  #   /usr/sbin/named
  # 3 processes have profiles defined.
  # 3 processes are in enforce mode.
  #   /usr/sbin/nginx (1234)
  #   /usr/sbin/mysqld (5678)
```

```
# ── Modes ──────────────────────────────────────────────────────────
# enforce  = violations denied AND logged
# complain = violations logged ONLY (like SELinux permissive)
# disabled = no enforcement, no logging

# ── Profile location ───────────────────────────────────────────────
ls /etc/apparmor.d/
# usr.bin.firefox
# usr.sbin.nginx
# usr.sbin.mysqld
# usr.lib.snapd.snap-confine

# ── Example profile: nginx ─────────────────────────────────────────
cat /etc/apparmor.d/usr.sbin.nginx
# #include <tunables/global>
#
# /usr/sbin/nginx {
#   #include <abstractions/base>
#   #include <abstractions/nameservice>
#
#   capability setgid,
#   capability setuid,
#   capability dac_override,
#
#   /var/www/html/** r,        ← read all files under /var/www/html
#   /var/log/nginx/*.log w,    ← write to nginx log files
#   /etc/nginx/** r,           ← read nginx config
#   /run/nginx.pid rw,         ← pid file: read and write
#   /tmp/** rw,                ← temp directory
#
#   deny /etc/shadow r,        ← explicitly deny shadow file
#   deny /home/** rw,          ← deny home directories
# }

# ── Mode management ────────────────────────────────────────────────

# Put nginx profile in complain mode (safe debugging)
sudo aa-complain /usr/sbin/nginx

# Put nginx profile in enforce mode
sudo aa-enforce /usr/sbin/nginx

# Disable a profile
sudo aa-disable /usr/sbin/nginx

# Reload all profiles
sudo systemctl reload apparmor
sudo apparmor_parser -r /etc/apparmor.d/usr.sbin.nginx

# ── Generating profiles ────────────────────────────────────────────

# aa-genprof: wizard that generates a profile for a program
sudo aa-genprof /usr/local/bin/myapp
```

```
# Run the app → aa-genprof captures what it accesses
# Answer questions about access → profile generated

# aa-logprof: update profile based on recent denials
sudo aa-logprof
# Reads /var/log/syslog or /var/log/kern.log for apparmor denials
# Presents each denial, lets you add to profile

# ── Checking AppArmor denials ──────────────────────────────────────

# AppArmor logs to syslog / audit log
grep "apparmor=\"DENIED\"" /var/log/syslog | tail -20
# Mar 10 14:32:01 server01 audit[1234]:
# type=1400 audit(1710012345.678:890): apparmor="DENIED"
# operation="open" profile="/usr/sbin/nginx" name="/etc/shadow"
# pid=1234 comm="nginx" requested_mask="r" denied_mask="r"

grep "apparmor" /var/log/kern.log | grep DENIED | tail -20
```

🔷 SELinux vs AppArmor comparison

```
┌──────────────────┬──────────────────────────┬──────────────────────────┐
│ Feature          │ SELinux                  │ AppArmor                 │
├──────────────────┼──────────────────────────┼──────────────────────────┤
│ Access model     │ Label-based (contexts)   │ Path-based               │
│ Granularity      │ Very fine (type enforcement│ Moderate               │
│ Learning curve   │ Steep                    │ Gentle                   │
│ Default on       │ RHEL, CentOS, Fedora     │ Ubuntu, Debian, SUSE     │
│ Policy creation  │ audit2allow (complex)    │ aa-genprof (guided)      │
│ Troubleshooting  │ ausearch + audit2allow   │ aa-logprof               │
│ Container use    │ Docker --security-opt    │ Docker --security-opt    │
│ Network labels   │ Yes (port types)         │ Limited                  │
│ MLS support      │ Yes                      │ No                       │
└──────────────────┴──────────────────────────┴──────────────────────────┘
```

🔷 Short crisp interview answer

> "SELinux and AppArmor both implement MAC — Mandatory Access Control — a second permission layer the kernel enforces regardless of file ownership. SELinux uses labels (contexts) on every file, process, and port: nginx runs with type `httpd_t` and can only access files with `httpd_sys_content_t` context. AppArmor uses path-based profiles: you list exactly which paths a process can access. Both have an 'enforcing' mode that blocks violations and a 'permissive/complain' mode that only logs them — I always start in permissive mode when adding a new profile. For SELinux, my debugging flow is: check `ausearch -m AVC`, use `audit2allow` to understand the denial, fix context with `semanage fcontext` + `restorecon`."

8.5 Linux Capabilities — Replacing Setuid Root

🔷 What capabilities are

Traditional Unix is binary: a process either runs as root (all-powerful) or as a regular user (restricted). **Capabilities** split root's omnipotence into **37 distinct privileges** that can be granted independently. This follows the principle of least privilege — give a process exactly what it needs, nothing more.

🔷 The problem capabilities solve

```
Old world:
  ping needs to send raw ICMP packets (requires raw socket)
  raw sockets require root privilege
  Therefore: ping is setuid root (-rwsr-xr-x)
  But: setuid root ping = ping runs as FULL ROOT
  Security risk: any bug in ping = root exploit

With capabilities:
  Give ping CAP_NET_RAW capability only
  ping can open raw sockets
  ping CANNOT read /etc/shadow, change other processes, load kernel
modules
  A ping exploit = only loses raw socket privilege, nothing else

ls -la $(which ping)
# -rwxr-xr-x 1 root root 72776 Mar 10 2026 /usr/bin/ping
#  No setuid bit!

getcap $(which ping)
# /usr/bin/ping cap_net_raw=ep
#                ___________
#                the capability: effective + permitted
```

🔷 Important capabilities

```
# Network capabilities
# CAP_NET_ADMIN     → configure network interfaces, routes, iptables, tc
# CAP_NET_BIND_SERVICE → bind to ports < 1024 (normally root-only)
# CAP_NET_RAW       → raw sockets, packet sniffing
# CAP_NET_BROADCAST → broadcast packets

# File capabilities
# CAP_DAC_OVERRIDE  → bypass file permission checks (dangerous!)
# CAP_DAC_READ_SEARCH → bypass read/execute permission checks on dirs
# CAP_CHOWN         → change file ownership
# CAP_FSETID        → set setuid/setgid bits
# CAP_SETUID        → change UID (set arbitrary UID)
# CAP_SETGID        → change GID

# System capabilities
# CAP_SYS_ADMIN     → MANY things: mount, swapon, sethostname, etc. (too
broad!)
# CAP_SYS_PTRACE    → trace processes (strace, gdb)
# CAP_SYS_BOOT      → reboot, kexec
# CAP_SYS_TIME      → set system clock
# CAP_KILL          → send signals to any process
# CAP_IPC_LOCK      → lock memory (mlock), important for databases
# CAP_SYS_NICE      → set nice/scheduling priority
# CAP_SYS_RAWIO     → raw I/O (iopl, ioperm)
# CAP_MKNOD         → create device files
# CAP_AUDIT_WRITE   → write to audit log
# CAP_SETPCAP       → set/drop capabilities for other processes

# Show all capabilities
cat /proc/self/status | grep Cap
# CapInh: 0000000000000000   ← inheritable
# CapPrm: 0000000000000000   ← permitted
# CapEff: 0000000000000000   ← effective (actually in use)
# CapBnd: 000001ffffffffff   ← bounding set (maximum allowed)
# CapAmb: 0000000000000000   ← ambient (passed to children)

# Decode capability bitmask
capsh --decode=000001ffffffffff
```

🔷 `getcap` and `setcap` — File capabilities

```
# ── View capabilities ──────────────────────────────────────────

# Check specific binary
getcap /usr/bin/ping
# /usr/bin/ping cap_net_raw=ep

# Check all files in a directory
getcap -r /usr/bin/ 2>/dev/null
```

```
# /usr/bin/ping cap_net_raw=ep
# /usr/bin/mtr cap_net_raw,cap_net_admin=eip

# Capability sets on files:
# e = effective (capability is active when program runs)
# p = permitted (capability can be activated)
# i = inheritable (passed to child processes)
# Most common: =ep (effective + permitted)

# ── Set capabilities ──────────────────────────────────────────

# Allow myserver to bind to port 80 without root
sudo setcap cap_net_bind_service=ep /usr/local/bin/myserver

# Allow a tool to do raw packet capture
sudo setcap cap_net_raw=ep /usr/local/bin/mytool

# Allow Java app to bind low ports (set on JVM binary)
sudo setcap cap_net_bind_service=ep /usr/lib/jvm/java-11/bin/java

# Multiple capabilities
sudo setcap cap_net_bind_service,cap_net_raw=ep /usr/local/bin/myapp

# ── Remove capabilities ───────────────────────────────────────

sudo setcap -r /usr/local/bin/myserver   # Remove all capabilities

# ── Verify ────────────────────────────────────────────────────

getcap /usr/local/bin/myserver
# /usr/local/bin/myserver cap_net_bind_service=ep
```

🔷 Process capabilities with `capsh`

```
# Show current process capabilities
capsh --print
# Current: =
# Bounding set: cap_chown,cap_dac_override,...,cap_block_suspend,...
# Securebits: 00/0x0/1'b0

# Run command with specific capabilities only
sudo capsh --caps="cap_net_bind_service+eip" -- -c
"/usr/local/bin/myserver"

# Drop capabilities from current process
capsh --drop=cap_sys_admin -- -c "id"

# ── /proc//status capability fields ──────────────────────────

cat /proc/1234/status | grep Cap
```

```
# CapInh: 0000000000000000
# CapPrm: 0000000000000400   ← cap_net_bind_service
# CapEff: 0000000000000400
# CapBnd: 000001ffffffffff
# CapAmb: 0000000000000000

# Decode:
python3 -c "
import struct
value = 0x0000000000000400
caps = []
cap_names = {9: 'CAP_KILL', 10: 'CAP_NET_BIND_SERVICE', 13: 'CAP_NET_RAW',
             21: 'CAP_SYS_ADMIN', 12: 'CAP_NET_ADMIN'}
for bit in range(64):
    if value & (1 << bit):
        print(f'bit {bit}: {cap_names.get(bit, \"unknown\")}')
"
```

🔷 Capabilities in Docker and Kubernetes

```
# ── Docker: capability management ──────────────────────────────────

# Docker drops these capabilities by default:
# CAP_AUDIT_WRITE, CAP_SETPCAP, CAP_NET_ADMIN, CAP_SYS_ADMIN...
# (Docker defaults are reasonable but not minimal)

# Add a capability
docker run --cap-add NET_ADMIN nginx

# Drop all capabilities, add only what's needed (best practice)
docker run --cap-drop ALL --cap-add NET_BIND_SERVICE myapp

# Run with NO capabilities at all
docker run --cap-drop ALL myapp

# ── Kubernetes: SecurityContext ────────────────────────────────────

# Pod spec:
apiVersion: v1
kind: Pod
spec:
  containers:
  - name: myapp
    securityContext:
      capabilities:
        drop:
        - ALL              # Drop everything
        add:
        - NET_BIND_SERVICE # Add only what's needed
      runAsNonRoot: true
```

```
        runAsUser: 1000
        allowPrivilegeEscalation: false
        readOnlyRootFilesystem: true
```

🔷 Short crisp interview answer

> "Linux capabilities split root privilege into 37 fine-grained privileges. Instead of making a binary setuid root, you grant it exactly the capability it needs: `setcap cap_net_bind_service=ep /usr/local/bin/server` lets the server bind to port 80 without any other root privilege. Key capabilities: `CAP_NET_BIND_SERVICE` for low ports, `CAP_NET_RAW` for raw sockets (ping, tcpdump), `CAP_NET_ADMIN` for network configuration. In containers, best practice is `--cap-drop ALL` then add only specific capabilities needed — this limits blast radius if the container is compromised."

⚠ Gotchas

```
# GOTCHA 1: CAP_SYS_ADMIN is the new root
# CAP_SYS_ADMIN covers: mount, ioctl, ptrace, cgroups, seccomp, keyring...
# It's used as a catch-all — granting it defeats the purpose of
capabilities
# Treat CAP_SYS_ADMIN as equivalent to root

# GOTCHA 2: setcap strips setuid
# If a binary has setuid AND capabilities, setuid is ignored
# setcap automatically removes setuid bit for security

# GOTCHA 3: capabilities are lost across exec() by default
# Unless using ambient capabilities (Linux 4.3+) or file capabilities
# A child process doesn't inherit parent's capabilities unless
specifically set

# GOTCHA 4: NFS and capabilities
# Files on NFS may not support extended attributes
# setcap on NFS files may silently fail or not work
# Use capabilities on binaries on local filesystems only

# GOTCHA 5: CAP_DAC_OVERRIDE is essentially full file access
# Bypasses ALL discretionary access control
# Almost as dangerous as full root for data access purposes
```

8.6 Seccomp — Syscall Filtering (Used by Docker/K8s)

🔷 What Seccomp is

Seccomp (Secure Computing Mode) filters the **system calls** a process is allowed to make. A process that only serves HTTP has no legitimate reason to call `ptrace()`, `reboot()`, or `keyctl()`. Seccomp lets you block those dangerous syscalls — even if an attacker achieves code execution, they can't use blocked syscalls.

🔷 How Seccomp works

```
Process makes syscall:
         │
         ▼
   Kernel entry point
         │
         ▼
   Is Seccomp filter set for this process?
         │
     YES │
         ▼
   BPF filter program evaluates syscall:
     - syscall number
     - arguments (optionally)
         │
     ┌───┴──────────────────────────────────┐
     │                                      │
  ALLOW                KILL_PROCESS    RETURN_ERRNO
  (syscall proceeds)   (SIGSYS sent)   (return error)
                       (process dies)  (looks like failure)
```

🔷 Seccomp modes

```
# Mode 1: strict — only read, write, exit, sigreturn allowed
# Almost nothing works in strict mode
# Used by: minimal containers, sandboxed processes

# Mode 2: filter — BPF-based custom filter
# Much more flexible — specify exact allowed/denied syscalls
# Used by: Docker, Kubernetes, Chrome, Firefox, Android

# Check if a process has seccomp
cat /proc/1234/status | grep Seccomp
# Seccomp: 2
# 0 = disabled
# 1 = strict mode
# 2 = filter mode (BPF filter applied)

# Check current process
grep Seccomp /proc/self/status
```

🔷 Docker's default Seccomp profile

```
# Docker applies a default seccomp profile that blocks ~44 dangerous
syscalls

# View Docker's default seccomp profile
cat /etc/docker/seccomp.json | python3 -m json.tool | head -50

# Syscalls blocked by default Docker seccomp:
# acct          → process accounting (root only anyway)
# add_key       → kernel keyring operations
# bpf           → load BPF programs (container escape risk)
# clock_adjtime → adjust system clock
# clone         → used with dangerous flags
# create_module → load kernel modules
# delete_module → unload kernel modules (kernel modification)
# finit_module  → load kernel module from fd
# get_kernel_syms
# init_module   → insert kernel module (critical!)
# ioperm        → raw hardware I/O port access
# iopl          → change I/O privilege level
# kcmp          → compare kernel structures (info leak)
# kexec_file_load → replace running kernel (!)
# kexec_load    → replace running kernel (!)
# keyctl        → kernel keyring management
# lookup_dcookie
# mbind         → NUMA memory policy
# mount         → mount filesystems
# move_pages    → NUMA page movement
# name_to_handle_at → file handle (container escape vector)
# nfsservctl    → NFS server operations
# open_by_handle_at → open by handle (container escape!)
# perf_event_open → performance monitoring (info leak risk)
# personality   → change process execution domain
# pivot_root    → change root filesystem
# process_vm_readv/writev → read/write another process's memory
# ptrace        → trace/debug processes (info leak, exploit)
# query_module  → query kernel module info
# quotactl      → manage disk quotas
# reboot        → reboot/halt/poweroff
# set_mempolicy → NUMA memory policy
# setns         → change namespace (container escape)
# settimeofday  → set system time
# stime         → set system time
# swapoff/swapon → manage swap
# _sysctl       → configure kernel parameters
# syslog        → read kernel log (info leak)
# umount2       → unmount filesystem
# unshare       → create new namespaces
# uselib        → use shared library (legacy)
# userfaultfd   → user-space fault handling (exploit primitive)
```

```
# ustat         → filesystem statistics (obsolete)
# vm86/vm86old  → 8086 virtual mode

# ── Custom seccomp profile ──────────────────────────────────────

cat /tmp/strict-profile.json
{
    "defaultAction": "SCMP_ACT_ERRNO",  ← block all syscalls by default
    "syscalls": [
        {
            "names": [
                "read", "write", "open", "close",
                "stat", "fstat", "lstat", "poll",
                "lseek", "mmap", "mprotect", "munmap",
                "brk", "rt_sigaction", "rt_sigprocmask",
                "ioctl", "access", "pipe", "select",
                "sched_yield", "mremap", "msync", "mincore",
                "madvise", "shmget", "shmat", "shmctl",
                "dup", "dup2", "pause", "nanosleep",
                "getitimer", "alarm", "setitimer",
                "getpid", "sendfile", "socket", "connect",
                "accept", "sendto", "recvfrom",
                "sendmsg", "recvmsg", "shutdown", "bind",
                "listen", "getsockname", "getpeername",
                "socketpair", "setsockopt", "getsockopt",
                "clone", "fork", "vfork", "execve",
                "exit", "wait4", "kill", "uname",
                "fcntl", "flock", "fsync", "fdatasync",
                "truncate", "ftruncate", "getdents",
                "getcwd", "chdir", "fchdir",
                "getuid", "getgid", "geteuid", "getegid",
                "exit_group", "futex", "set_tid_address",
                "openat", "getdents64"
            ],
            "action": "SCMP_ACT_ALLOW"   ← allow these specific syscalls
        }
    ]
}

# Run container with custom seccomp profile
docker run --security-opt seccomp=/tmp/strict-profile.json myapp

# Run WITHOUT seccomp (for debugging — never in production)
docker run --security-opt seccomp=unconfined myapp

# ── Seccomp actions ─────────────────────────────────────────────
# SCMP_ACT_ALLOW       → allow the syscall
# SCMP_ACT_ERRNO       → return EPERM error (polite denial)
# SCMP_ACT_KILL        → kill process with SIGSYS (harsh)
# SCMP_ACT_KILL_PROCESS → kill entire process (not just thread)
# SCMP_ACT_TRAP        → send SIGSYS (for debugging)
# SCMP_ACT_TRACE       → notify tracer (ptrace)
# SCMP_ACT_LOG         → allow but log (audit mode)
```

🔷 Using `seccomp` in code

```
// C: apply seccomp filter directly
#include

scmp_filter_ctx ctx = seccomp_init(SCMP_ACT_KILL);  // default: kill
seccomp_rule_add(ctx, SCMP_ACT_ALLOW, SCMP_SYS(read), 0);
seccomp_rule_add(ctx, SCMP_ACT_ALLOW, SCMP_SYS(write), 0);
seccomp_rule_add(ctx, SCMP_ACT_ALLOW, SCMP_SYS(exit_group), 0);
seccomp_load(ctx);
// Now process can only read, write, exit
```

🔷 Kubernetes Seccomp

```
# Pod spec with seccomp profile
apiVersion: v1
kind: Pod
spec:
  securityContext:
    seccompProfile:
      type: RuntimeDefault    # Use container runtime's default profile
      # type: Localhost       # Use custom profile from node
      # type: Unconfined      # No seccomp (unsafe)
  containers:
  - name: myapp
    securityContext:
      allowPrivilegeEscalation: false
```

🔷 Short crisp interview answer

> "Seccomp filters the exact system calls a process is allowed to make using a BPF program evaluated at syscall entry. Docker applies a default seccomp profile that blocks ~44 dangerous syscalls — `ptrace`, `mount`, `reboot`, `init_module` (load kernel modules), `open_by_handle_at` (container escape vector). This limits what an attacker can do even after achieving code execution. The profile format is JSON: `defaultAction: SCMP_ACT_ERRNO` blocks everything, then you whitelist allowed syscalls. In Kubernetes, `seccompProfile: type: RuntimeDefault` applies the container runtime's curated profile."

8.7 Audit Framework — `auditd`, Tracking Privileged Access

🔷 What auditd is

The Linux Audit Framework is a **kernel-level security logging system**. Unlike application logs that can be tampered with by a compromised application, audit events are written to the kernel ring buffer and then to the audit log by a dedicated daemon — before the application can intercept them.

🔷 How auditd works

```
System call or kernel event occurs
              │
              ▼
    Kernel audit subsystem
    (checks if event matches any audit rules)
              │
    If matches:
              │
              ▼
    Event written to kernel audit buffer
    (/proc/audit — kernel ring buffer)
              │
              ▼
    auditd daemon reads from kernel buffer
              │
              ▼
    audisp plugins dispatch events:
    ├── /var/log/audit/audit.log (raw)
    ├── syslog / journald
    ├── remote logging (audisp-remote)
    └── SIEM / Splunk / ELK

  Why this matters for security:
  Even if attacker compromises an application:
  ✓ Cannot delete past audit records (already in file)
  ✓ Cannot prevent auditd from logging (kernel-level)
  ✗ Can stop auditd service (if they have root)
  → Use immutable audit log (auditd -e 2) or remote logging
```

🔷 Installing and configuring auditd

```
# Install
sudo apt install auditd audispd-plugins
sudo yum install audit audit-libs

# Enable and start
sudo systemctl enable --now auditd

# Configuration
```

```
cat /etc/audit/auditd.conf
# log_file = /var/log/audit/audit.log
# log_format = ENRICHED      ← human-readable vs RAW
# max_log_file = 50          ← max log file size in MB
# max_log_file_action = ROTATE  ← what to do when full
# num_logs = 10              ← keep 10 rotated files
# space_left = 75            ← warn when disk < 75MB
# space_left_action = SYSLOG ← log warning
# admin_space_left = 50      ← critical disk threshold
# admin_space_left_action = SUSPEND ← stop logging if disk critically low
# disk_full_action = SUSPEND
# disk_error_action = SYSLOG

# Apply config change
sudo systemctl restart auditd
# ⚠ auditd cannot be reloaded with kill -HUP — must restart
# Or: sudo auditctl -R /etc/audit/rules.d/audit.rules (reload rules only)
```

🔷 `auditctl` — Managing audit rules

```
# ── View current rules ──────────────────────────────────────────
sudo auditctl -l
# -a always,exit -F arch=b64 -S open -F key=file_access
# -w /etc/passwd -p rwa -k identity_changes

# ── Watch files (filesystem rules) ──────────────────────────────

# Monitor /etc/passwd for any access
sudo auditctl -w /etc/passwd -p rwa -k identity_changes
# -w = watch this file
# -p = permissions to watch: r=read, w=write, a=attribute change,
x=execute
# -k = key/tag for searching logs

# Monitor /etc/sudoers changes
sudo auditctl -w /etc/sudoers -p wa -k sudo_changes

# Monitor /etc/shadow reads (password hash access)
sudo auditctl -w /etc/shadow -p rwa -k shadow_access

# Watch entire directory
sudo auditctl -w /etc/ssh/ -p rwa -k ssh_config

# Watch binary execution
sudo auditctl -w /usr/bin/wget -p x -k download_tool
sudo auditctl -w /usr/bin/curl -p x -k download_tool

# ── System call rules ───────────────────────────────────────────

# Log all successful su/sudo attempts
```

```
sudo auditctl -a always,exit -F arch=b64 -S execve \
    -F path=/usr/bin/sudo -F perm=x -F auid>=1000 \
    -F auid!=4294967295 -k privilege_escalation

# Log all privileged command execution
sudo auditctl -a always,exit -F arch=b64 -S execve \
    -F euid=0 -F auid>=1000 -F auid!=4294967295 \
    -k root_commands

# Log all logins
sudo auditctl -a always,exit -F arch=b64 -S open \
    -F path=/var/log/lastlog -F perm=r -k logins

# Log mount/umount operations
sudo auditctl -a always,exit -F arch=b64 \
    -S mount -S umount2 -k mounts

# Log file deletion
sudo auditctl -a always,exit -F arch=b64 \
    -S unlink -S unlinkat -S rename -S renameat \
    -F auid>=1000 -k file_deletion

# Log changes to user/group files
sudo auditctl -w /etc/group -p wa -k identity_changes
sudo auditctl -w /etc/gshadow -p wa -k identity_changes
sudo auditctl -w /etc/security/opasswd -p wa -k identity_changes

# Monitor network configuration changes
sudo auditctl -a always,exit -F arch=b64 \
    -S sethostname -S setdomainname -k network_changes
sudo auditctl -w /etc/hosts -p wa -k network_changes
sudo auditctl -w /etc/sysconfig/network -p wa -k network_changes

# ── Make rules permanent ──────────────────────────────────────────

# Temporary rules (lost on reboot): auditctl commands above
# Permanent: add to /etc/audit/rules.d/

cat /etc/audit/rules.d/audit.rules
# Delete all rules
-D
# Buffer size
-b 8192
# Failure mode (0=silent, 1=printk, 2=panic)
-f 1

# Watch identity files
-w /etc/passwd -p rwa -k identity_changes
-w /etc/shadow -p rwa -k shadow_access
-w /etc/sudoers -p rwa -k sudo_changes
-w /etc/sudoers.d/ -p rwa -k sudo_changes

# Privileged commands
-a always,exit -F arch=b64 -S execve -F euid=0 -F auid>=1000 \
```

```
  -F auid!=4294967295 -k root_commands

# Load rules
sudo augenrules --load
# Or: sudo auditctl -R /etc/audit/rules.d/audit.rules

# ── Immutable mode ──────────────────────────────────────────
# Lock rules so they can't be changed until reboot
sudo auditctl -e 2    # Immutable
# Now: auditctl -D  (delete rules) → returns "Operation not permitted"
# Can only be changed by rebooting

# ── View audit status ───────────────────────────────────────
sudo auditctl -s
# enabled 1
# failure 1
# pid 1234
# rate_limit 0
# backlog_limit 8192
# lost 0
# backlog 0
# backlog_wait_time 60000
# loginuid_immutable 0
```

🔷 `ausearch` — Query audit logs

```
# ── Search by key ───────────────────────────────────────────

sudo ausearch -k sudo_changes
sudo ausearch -k shadow_access
sudo ausearch -k root_commands --start today

# ── Search by time ──────────────────────────────────────────

sudo ausearch -ts today           # Today's events
sudo ausearch -ts recent          # Last 10 minutes
sudo ausearch -ts "03/10/2026 14:00:00" -te "03/10/2026 15:00:00"
sudo ausearch -ts yesterday -te today

# ── Search by user ──────────────────────────────────────────

sudo ausearch -ua alice           # Events for user alice (by login UID)
sudo ausearch -ui 1000            # Events for UID 1000

# ── Search by type ──────────────────────────────────────────

sudo ausearch -m USER_LOGIN       # Login events
sudo ausearch -m USER_AUTH        # Authentication events
sudo ausearch -m AVC              # SELinux denials
sudo ausearch -m SYSCALL          # Syscall events
```

```
sudo ausearch -m EXECVE           # Program execution events

# ── Combined search ──────────────────────────────────────────

# Who ran what as root today?
sudo ausearch -k root_commands -ts today | aureport --exe --summary

# All failed logins
sudo ausearch -m USER_AUTH --success no

# What did alice do?
sudo ausearch -ua alice -ts today | aureport --file --summary

# ── ausearch with aureport ───────────────────────────────────

# aureport: generate summary reports from audit logs

# Summary of all events
sudo aureport

# Summary of executions
sudo aureport --exe --summary

# Failed authentication report
sudo aureport --auth --summary --failed

# Anomaly report
sudo aureport --anomaly

# Login/logout report
sudo aureport --login

# File access report
sudo aureport --file --summary | head -20

# ── Practical investigation ──────────────────────────────────

# "Who changed /etc/sudoers?"
sudo ausearch -k sudo_changes -f /etc/sudoers | grep "type=SYSCALL"

# "What commands did alice run as root?"
sudo ausearch -k root_commands -ua alice -ts today

# "Were there any failed sudo attempts?"
sudo ausearch -m USER_AUTH -m USER_CMD --success no -ts today

# "Did anyone access /etc/shadow today?"
sudo ausearch -k shadow_access -ts today
```

🔷 Reading audit log format

```
# Raw audit log example:
cat /var/log/audit/audit.log | tail -20

# A sudo command event creates multiple records linked by same timestamp:

# Record 1: SYSCALL
# type=SYSCALL msg=audit(1710012345.678:890):
#   arch=c000003e      ← x86_64
#   syscall=59         ← execve()
#   success=yes
#   exit=0
#   a0=55a1b2c3d4e5    ← arg0 (pointer to executable path)
#   a1=55a1b2c3d4f6    ← arg1 (argv[])
#   a2=7fff12345678    ← arg2 (envp[])
#   items=3
#   ppid=1234          ← parent PID (the shell)
#   pid=5678           ← this process
#   auid=1000          ← audit UID = alice's UID (set at login, doesn't
change with su/sudo)
#   uid=0              ← current UID = 0 (running as root via sudo)
#   gid=0
#   euid=0             ← effective UID = 0
#   suid=0
#   fsuid=0
#   egid=0
#   sgid=0
#   fsgid=0
#   tty=pts0
#   ses=42             ← session ID
#   comm="systemctl"   ← command name
#   exe="/usr/bin/systemctl"  ← full path
#   subj=unconfined_u:unconfined_r:unconfined_t:s0  ← SELinux context
#   key="root_commands"  ← our rule's key tag

# Record 2: EXECVE (the actual arguments)
# type=EXECVE msg=audit(1710012345.678:890):
#   argc=3
#   a0="systemctl"
#   a1="restart"
#   a2="nginx"

# Record 3: PATH (the binary's path and inode)
# type=PATH msg=audit(1710012345.678:890):
#   item=0 name="/usr/bin/systemctl" inode=12345 dev=08:01
#   mode=0100755 ouid=0 ogid=0 rdev=00:00 nametype=NORMAL cap_fp=0
cap_fi=0 cap_fe=0

# KEY INSIGHT: auid (audit UID) is set at LOGIN and NEVER changes
# Even with sudo, su, or su - root → auid stays as original login user
# This is how you trace WHO actually did something, even after escalation
```

🔷 Short crisp interview answer

> "auditd is kernel-level security logging — events are captured by the kernel before any userspace can intercept them. I set audit rules with `auditctl -w /etc/sudoers -p wa -k sudo_changes` (watch for write/attribute changes) and syscall rules with `-a always,exit -F arch=b64 -S execve -F euid=0 -k root_commands` to log every command run as root. The key field is `auid` — the audit UID set at login that never changes even through sudo or su, so I can trace actions back to the original user. `ausearch -k sudo_changes -ts today` searches by my key tag, and `aureport --auth --summary` gives aggregated authentication reports."

⚠ Gotchas

```
# GOTCHA 1: auid=4294967295 means unset (untracked sessions)
# Kernel threads, cron jobs, and some daemons don't have a login session
# auid=4294967295 = 0xFFFFFFFF = AUDIT_LOGINUID_UNSET
# Exclude from human-tracking rules: -F auid!=4294967295

# GOTCHA 2: Audit log fills disk and stops system (by design!)
# admin_space_left_action = SUSPEND → auditd stops logging
# disk_full_action = SUSPEND → on full disk, auditd suspends
# If configured to halt: disk_full_action = HALT → system halts on full
disk!
# Monitor disk space and rotate logs

# GOTCHA 3: High rule count impacts performance
# Every syscall checks all rules → too many rules = syscall overhead
# Benchmark: auditctl -s to see backlog_limit and lost counts
# lost > 0 = audit events are being dropped!

# GOTCHA 4: -F arch=b64 and 32-bit compat
# On 64-bit system: 32-bit processes use different syscall numbers
# Always add both arch rules:
# -a always,exit -F arch=b64 -S execve -k root_commands
# -a always,exit -F arch=b32 -S execve -k root_commands

# GOTCHA 5: auditd restart kills rules
sudo systemctl restart auditd   # Reloads auditd.conf but KEEPS rules
sudo auditctl -D                # Deletes all rules!
# Rules loaded by: augenrules --load or auditctl -R rulefile
```

🏆 Category 8 — Complete Mental Model

```
LINUX SECURITY LAYER MODEL
═══════════════════════════
```

```
      Application
           │
      ┌────▼──────────────────────────────────────────┐
      │  sudo / PAM                                   │
      │  Who can run what as whom? (authentication,   │
      │  authorization, resource limits, MFA)         │
      └───────────────────────────────────────────────┘
           │
      ┌────▼──────────────────────────────────────────┐
      │  DAC: Standard Unix Permissions + ACLs        │
      │  Owner/group/other + per-user/group entries   │
      └───────────────────────────────────────────────┘
           │
      ┌────▼──────────────────────────────────────────┐
      │  MAC: SELinux / AppArmor                      │
      │  Process context limits what files/ports it   │
      │  can access regardless of Unix permissions    │
      └───────────────────────────────────────────────┘
           │
      ┌────▼──────────────────────────────────────────┐
      │  Linux Capabilities                           │
      │  Process has only the specific privileges it  │
      │  needs (not full root)                        │
      └───────────────────────────────────────────────┘
           │
      ┌────▼──────────────────────────────────────────┐
      │  Seccomp                                      │
      │  Process can only call specific syscalls      │
      │  (even if code is compromised)                │
      └───────────────────────────────────────────────┘
           │
      ┌────▼──────────────────────────────────────────┐
      │  auditd                                       │
      │  All privileged actions logged at kernel level │
      │  Forensic trail, compliance, incident response │
      └───────────────────────────────────────────────┘

DEFENSE IN DEPTH:
  Each layer independent — compromise one, others hold.
  Attacker needs to bypass ALL layers to achieve full compromise.
```

🔷 Quick Reference

```
# sudo — check what a user can do
sudo -l -U alice

# PAM — check account lockout status
faillock --user alice

# ACL — view/set permissions
```

```
getfacl /path/to/file
setfacl -m u:bob:r-- /path/to/file
setfacl -m d:u:bob:r-- /path/to/dir   # default for new files

# SELinux — check and fix context
getenforce                             # Enforcing/Permissive/Disabled
ls -Z /path/to/file                   # View context
ausearch -m AVC -ts recent            # View denials
semanage fcontext -a -t httpd_sys_content_t "/data/www(/.*)?"
restorecon -Rv /data/www/

# AppArmor — check and manage profiles
aa-status
aa-complain /usr/sbin/nginx           # Switch to complain mode
aa-enforce /usr/sbin/nginx            # Switch to enforce mode
aa-logprof                            # Update profile from logs

# Capabilities — view and set
getcap /usr/bin/ping
setcap cap_net_bind_service=ep /usr/local/bin/server
setcap -r /usr/local/bin/server       # Remove capabilities

# Seccomp — check process
grep Seccomp /proc/1234/status        # 0=off, 1=strict, 2=filter

# auditd — search logs
ausearch -k sudo_changes -ts today
ausearch -m USER_AUTH --success no    # Failed auths
aureport --exe --summary              # Execution summary
```

⚠ Master Gotcha List

#	Gotcha	Reality
1	Edit `/etc/sudoers` directly	Always use `visudo` — syntax error = lockout
2	`sudo vim` is safe	vim can run shell commands — `sudoedit` instead
3	New files in ACL dir inherit ACL	They DON'T — set default ACL with `d:` prefix
4	`chmod` respects ACL mask	`chmod` resets the mask, may silently drop ACL permissions
5	PAM misconfiguration is survivable	Broken pam.d/sshd = can't SSH in — test with second session
6	SELinux enforcing breaks new deployments	Start permissive, use `ausearch + audit2allow` to build policy
7	`CAP_SYS_ADMIN` is fine-grained	It's effectively root — covers mount, ptrace, cgroups, etc.
8	Seccomp default = secure enough	Docker's default still allows ~300 syscalls — build custom for critical apps

#	Gotcha	Reality
9	auditd `auid=4294967295` is a user	It means no login session — kernel threads, cron, daemons
10	Audit logs are tamper-proof	auditd can be stopped by root — use immutable mode + remote logging
11	AppArmor profile = complete protection	Path-based profiles can be bypassed via symlinks if not careful
12	`setenforce 0` is temporary	`/etc/selinux/config` for permanent — and disabled needs relabel + reboot

🔥 Top Interview Questions

Q1: How would you grant a developer permission to restart nginx without giving them full sudo?

> Edit sudoers with `visudo` and add: `alice ALL=(root) NOPASSWD: /usr/bin/systemctl restart nginx, /usr/bin/systemctl status nginx`. Use the full path to systemctl, specify the exact service name in the argument, and use NOPASSWD if it's for automation. Drop the file in `/etc/sudoers.d/alice` for cleanliness. The action is logged to `/var/log/auth.log` automatically.

Q2: SELinux is blocking nginx from serving files from `/data/www`. How do you fix it without disabling SELinux?

> First confirm it's SELinux: `setenforce 0` temporarily and test — if it works, SELinux is the culprit. Check the context: `ls -Z /data/www/` — it'll show `default_t` instead of `httpd_sys_content_t`. Fix it: `semanage fcontext -a -t httpd_sys_content_t "/data/www(/.*)?"` then `restorecon -Rv /data/www/`. Re-enable enforcing. Check for any remaining denials: `ausearch -m AVC -ts recent`.

Q3: What's the difference between capabilities and setuid?

> Setuid grants the entire set of root's privileges — any exploit in a setuid binary gives full root. Capabilities split root's power into 37 specific privileges. `ping` only needs `CAP_NET_RAW` to create raw sockets — with `setcap cap_net_raw=ep /usr/bin/ping` and no setuid bit, a ping exploit only compromises raw socket ability, nothing else. The principle of least privilege — give exactly what's needed, nothing more.

Q4: How does auditd track actions even after a user does `sudo su -`?

> The `auid` (audit UID) field is set at login time and stored in the kernel session record. It never changes regardless of subsequent `sudo`, `su`, or `su -` calls. Even if a user becomes root, `auid` still reflects their original login UID. This is how forensic investigation can trace "root ran this command" back to "alice logged in via SSH and escalated." `ausearch -ua alice` queries by auid, finding all their actions including post-escalation ones.

Q5: What syscalls does Docker's seccomp profile block and why?

> The most important blocks: `init_module`/`finit_module` (load kernel modules — direct kernel compromise), `ptrace` (read/write other processes' memory — inter-container attacks), `open_by_handle_at`/`name_to_handle_at` (container escape via filesystem handle bypassing mount namespace), `reboot` (halt/reboot host), `kexec_load` (replace running kernel), `setns` (enter another namespace — container escape), and `mount`/`pivot_root` (filesystem manipulation). These are all legitimate for the host but have no business use in a container and provide container escape primitives.

Q6: How do ACL default entries work and why do you need them?

> Regular ACL entries apply to the file or directory itself. Default ACL entries (set with `d:` prefix: `setfacl -m d:u:bob:r-- /shared/`) are stored on the directory and automatically applied to new files and subdirectories created inside it. Without default ACLs, alice creates `/shared/newfile.txt` and bob loses access to it because the ACL from the parent wasn't inherited. Default ACLs solve the "ongoing access" problem for collaborative directories.

This document covers all 7 topics in Category 8: Security & Permissions — from sudo internals and PAM authentication through ACLs, SELinux/AppArmor MAC enforcement, Linux capabilities, seccomp syscall filtering, and the auditd framework for compliance and forensics.